

Harbor + Terraform + Pipeline CI sécurisée

Déployer un registre d'images par Infrastructure as Code, sécuriser sa chaîne de livraison et la rendre observable.

IaC (Terraform/Helm) · CI sécurisée (lint·SAST·CVE·DAST·signature) · **Observabilité**

CONTEXTE

Objectif du projet

Mettre en place un **registre d'images Docker d'entreprise (Harbor)**, installé automatiquement et sécurisé de bout en bout.

Tout repose sur un mini-Kubernetes local (**k3s via k3d**) et un dépôt GitHub avec une pipeline qui s'exécute à chaque modification du code.

3 composantes imposées

Infrastructure as Code

Terraform · Helm · k3s

CI sécurisée (DevSecOps)

lint · SAST · CVE · DAST · signature

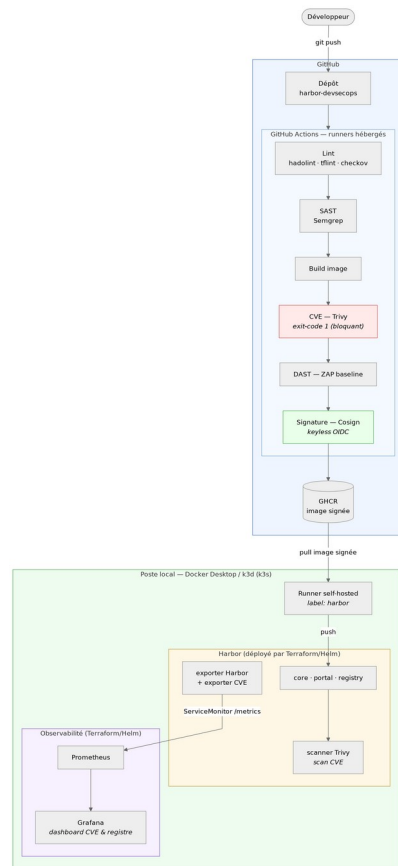
Observabilité

Prometheus · Grafana

Architecture

Le parcours, du code à Harbor :

1. Le développeur pousse son code sur GitHub.
2. GitHub Actions (runners hébergés) enchaîne les contrôles : lint → SAST → build → CVE → DAST → signature → GHCR.
3. Un runner self-hosted (poste local) tire l'image signée et la pousse dans Harbor.
4. Harbor la re-scanne ; Prometheus collecte les métriques ; Grafana les affiche.



Les trois piliers

1

Infrastructure as Code

Terraform pilote Helm pour déployer Harbor + la surveillance sur k3s.

Poids oral ≈ 25 %

2

CI sécurisée (DevSecOps)

5 portes bloquantes : lint, SAST, CVE, DAST, signature Cosign.

Poids oral ≈ 45 %

3

Observabilité

Harbor → Prometheus → Grafana : métriques du registre et CVE.

Poids oral ≈ 25 %

Terraform, Helm, k3s

k3s / k3d

Le cluster Kubernetes léger qui héberge tout (k3d = k3s dans Docker).

Helm

Le gestionnaire de paquets de Kubernetes : installe les applications via des charts.

Terraform

L'orchestrateur IaC : pilote Helm, suit l'état, gère l'ordre des dépendances.

Une seule commande

```
terraform init  
prépare (providers)  
terraform plan  
prévisualise le diff  
terraform apply  
déploie réellement
```

Idempotent : réappliquer ne casse rien.

main.tf : deux points à savoir

depends_on (l'ordre)

Impose l'ordre d'installation : **Prometheus AVANT Harbor**.

Pourquoi ? Harbor crée un **ServiceMonitor**, qui est une **CRD fournie par la stack Prometheus**. Sans cet ordre, la création échoue.

set_sensitive (les secrets)

Injecte les mots de passe (Harbor, Grafana) en les marquant **secrets** → **masqués** dans les logs et le plan Terraform.

Dans le fichier values, ils apparaîtraient en clair.

Les 5 portes de sécurité (bloquantes)

Porte	Outil	Ce qui est inspecté
Lint	hadolint · tflint · checkov	Qualité/sécurité du Dockerfile et de l'laC
SAST	Semgrep	Le code source, analysé sans l'exécuter
CVE	Trivy	Vulnérabilités connues des dépendances / OS
DAST	OWASP ZAP	L'application en cours d'exécution (attaque externe)
Signature	Cosign	Signe l'image : preuve d'origine (supply chain)

Chaque outil regarde un angle différent : le fichier, le code, les dépendances, l'appli vivante, l'authenticité.

SAST vs DAST, et « bloquant »

SAST — statique

Semgrep analyse le CODE SOURCE au repos, sans l'exécuter. Recherche des motifs dangereux.

DAST — dynamique

OWASP ZAP teste l'APPLI EN EXÉCUTION, de l'extérieur, comme un attaquant.

« Bloquant »

Trivy --exit-code 1
HIGH/CRITICAL corrigibles

→ code 1 = pipeline rouge, image non livrée.

Démo : Trivy sur l'app à jour → 0 (vert) ; sur python:3.9-bullseye → 1 (rouge, 7 CVE Critical).

Mnémon : SAST = Statique (code éteint) · DAST = Dynamique (appli allumée).

Signature Cosign « keyless »

Pourquoi signer ? Garantir l'origine et l'intégrité de l'image (sécurité de la chaîne d'approvisionnement / supply chain).

Keyless = sans clé privée

Rien à gérer ni à voler : pas de secret de signature.

Identité = le workflow GitHub

Prouvée par OIDC (le jeton du workflow).

Certificat éphémère : Fulcio

Émis pour la signature, puis expiré.

Preuve publique : Rekor

Journal de transparence infalsifiable.

cosign verify ... prouve que l'image vient bien de TON dépôt et de TON workflow.

Runners : hébergés vs self-hosted

Runners hébergés (cloud GitHub)

Contrôles sans état :

lint · SAST · build · CVE · DAST · signature → GHCR

Runner self-hosted (poste local)

Publication interne :

tire l'image de GHCR → la pousse dans Harbor → déclenche le scan.

Pourquoi ? Harbor est déployé en local, non exposé sur Internet : les runners cloud ne peuvent pas l'atteindre. C'est le patron standard en entreprise (déploiement vers l'infra interne par un runner du réseau privé).

Harbor → Prometheus → Grafana

1. Harbor

Expose /metrics + scanne les images (Trivy intégré)

2. ServiceMonitor

Dit à Prometheus : « viens collecter Harbor » (CRD)

3. Prometheus

Collecte et stocke les métriques (PromQL)

4. Grafana

Affiche le dashboard « Registre & CVE »

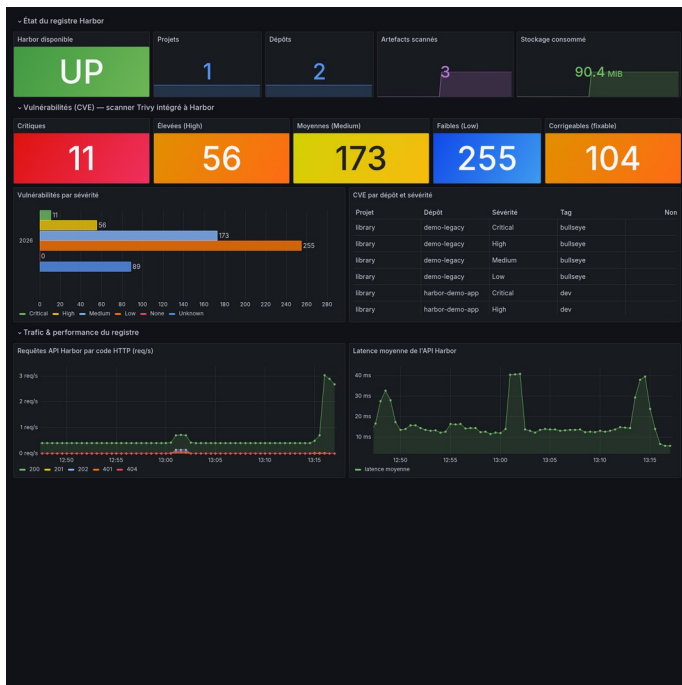
Exporter CVE maison

Les métriques natives de Harbor ne donnent pas le détail des CVE.

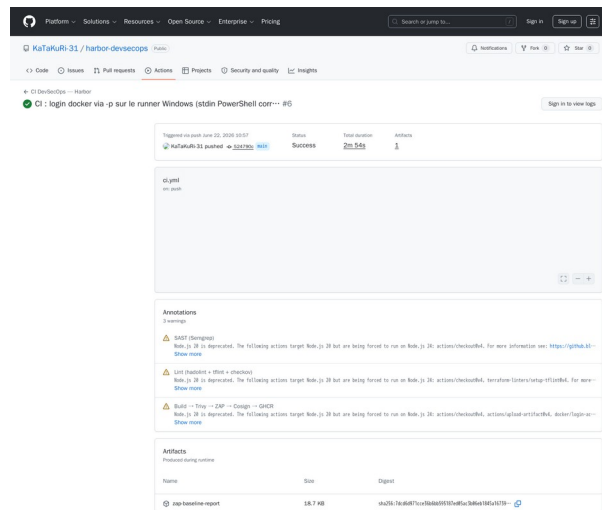
Un petit exporter interroge l'API Harbor et expose les CVE par sévérité → panneau « CVE » du dashboard.

DÉMONSTRATION

Preuves : dashboard & pipeline



Dashboard Grafana — CVE par sévérité & état du registre



GitHub Actions — pipeline au statut Success

Bilan

Livré

Harbor par Terraform/Helm sur k3s + pipeline CI bloquante verte + dashboard CVE.

À retenir 1

SAST = statique (code) · DAST = dynamique (appli en exécution).

À retenir 2

« Bloquant » = Trivy exit-code 1 sur les CVE HIGH/CRITICAL corrigeables.

À retenir 3

Runner self-hosted car Harbor est local, injoignable depuis le cloud GitHub.